

Momentan werden hier nur vereinzelte Gedankengänge festgehalten, die v.a. aufgrund fehlender Visualisierungen, Übungen und Unvollständigkeit eher für Lehrkräfte als für Studierende geeignet sind.

TODOs: Datenstrukturen, sachter Umstieg von Notizblock-Denkweise auf eine aktuelle Hochsprache, bekannte Algorithmen, Intelligenz (Problemlösung) ist auch nur Routine (Startsituation -> Anwendung von geeigneten Werkzeugen -> Zielsituation), Übungen, Übungen, Übungen in jedem Kapitel

Inhaltsverzeichnis

1	Natürliche Sprache präzisieren für Informatik	3
2	Notizblock – der Speicher	3
3	Datentypen und Hilfsfunktionen – Was steht in einer Zelle?	4
3.1	Einfache Datentypen (ein Zeichen pro Zelle)	5
3.2	Zusammengesetzte Typen (mehrere Zellen)	5
4	Tätigkeitenblatt – Abläufe als Bausteine	6
5	Prozessor – der Ausführer	7
5.1	1. Einfache Schreibvorgänge	7
5.2	2. Einfache Lesevorgänge	8
5.3	3. Ein einfacher Durchlauf	8
5.4	4. Eine Schleife mit Abfrage	8
6	Rechnen mit integers – ein erster Algorithmus	9
6.1	Grundidee	9
6.2	Beispiel: Addition mit Übertrag	9
6.3	Beispiel: Subtraktion mit Übertrag / Ausleihen	10
6.4	Warum ist das ein Notizblock-Maschinen-Schritt?	10
6.5	Erkenntnis	10
7	Hilfsblock – Hilfsvariablen	11
8	Laufzeitkomplexität: Schritte zählen	13
8.1	Warum Schritte zählen?	13
8.2	Beispiel 1: Lineare Suche im Notizblock	13
8.2.1	Startsituation:	13
8.2.2	Tätigkeit (rezeptartig):	13
8.2.3	Schritte zählen:	14
8.3	Beispiel 2: Binärsuche (im alphabetisch sortierten Notizblock)	14
8.3.1	Startsituation:	14
8.3.2	Tätigkeit:	14
8.3.3	Schritte zählen:	14
8.4	Vergleich der beiden Verfahren	15
8.5	Verbindung zur späteren Umsetzung	15

9 Sanfter Umstieg in eine Hochsprache (JavaScript)	15
9.1 Erster Übergang: Notizblock wird zur Matrix	15
9.2 Mini-Beispiel (gleicher Inhalt, zwei Schreibweisen)	16
9.3 Unsere Bücherei aus der alten Welt übertragen	16
9.4 Schleifendenken vom Notizblock zu JavaScript	17
10 Zusammenhang Informatik - Mathematik	17
10.0.1 Gleichungen über Zahlen	18
10.0.2 Gleichungen über nicht-numerische Objekte: Farben	18
10.0.3 Gleichungssysteme über bewegende Objekte	18
11 Schlussfolgerungen	19
11.1 Was ist eine Schlussfolgerung?	19
11.2 Beispiele aus der natürlichen Sprache	19
12 Beweisführung	20
12.1 Was ist eine Beweisführung?	20
12.2 Beispiel 1 – Sokrates	20
12.3 Beispiel 2 – Arithmetik: Quadratische Faktorisierung	20
12.3.1 einige arithmetische Werkzeuge	21
12.3.2 Ausgewähltes Werkzeug	21
12.3.3 Schlussfolgerung / Ableitung	21
12.4 Erkenntnis	21
12.5 Beispiel 3 - Teilbarkeit prüfen mit Polynom-Kongruenz	21
12.5.1 Werkzeuganwendung: Übersetzen in arithmetische Sprache	22
12.5.2 Werkzeuganwendung: Polynom-Kongruenz / Restklassen (Strukturerhalt des Terms)	22
12.5.3 Beweisführung: Prüfung aller Restklassen	23
12.6 Beispiel 4 - Prüfung der Geradzahligkeit	23
12.6.1 Geradzahligkeit prüfen mit Modulo 2	23
13 Tipps zur Didaktik	23
14 Wie kann man das Kochen lernen?	24

1 Natürliche Sprache präzisieren für Informatik



Wir Menschen nutzen unsere **natürliche Sprache** – zum Beispiel Deutsch oder Englisch –, um Sachverhalte zu beschreiben. Doch viele Sätze (und sogar einzelne Begriffe) können unterschiedlich verstanden werden. Für die Informatik und Mathematik ist das ein Problem, denn dort müssen Beschreibungen eindeutig sein. Deshalb verwendet man Programmiersprachen. Und bevor wir uns großen Sprachen widmen, beginnen wir mit einer sehr einfachen, klar definierten Sprache, mit der sich Inhalte präzise und ohne Mehrdeutigkeit beschreiben lassen.

2 Notizblock – der Speicher



Der Notizblock ist unser **Speicher**: Er enthält alle Informationen, die wir geordnet ablegen und wieder auslesen können. Wir nutzen **kariertes Papier**, auf dem wir Informationen an beliebigen Positionen notieren können. Jede Zelle hat eine Adresse [Zeile, Spalte] – genau wie beim handschriftlichen Schreiben auf kariertem Papier. Grundsätzlich kann man die Informationen genauso wie auf einem herkömmlichen Notizblock anordnen.

Beispiel für eine mögliche tabellarische Organisation:

```
# Kariertes Papier - Beispiel einer Tabellenstruktur
# (Man könnte die Informationen aber auch ganz anders anordnen)
# Jede Zeile steht für einen Eintrag, jede Spalte für ein Informationsfeld
# Wichtig: Wie beim karierten Zettel kommt jedes Zeichen (Buchstabe, Ziffer, Symbol) in eine eigene Zelle.
# Beispiel: Einkaufszettel (Mengen, Einheiten und Ladennamen zeichenweise pro Zelle)
```

	Sp.1	Sp.2	Sp.3	Sp.4	Sp.5	Sp.6	Sp.7	Sp.8	Sp.9	Sp.10	Sp.11
Zeile 1	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]	[1,7]	[1,8]	[1,9]	[1,10]	[1,11]
(Mehl)	MG1	MG2	EH1	EH2	EH3	(leer)	L1	L2	L3	L4	L5
	1	2	k	g			E	d	e	k	a
Zeile 2	[2,1]	[2,2]	[2,3]	[2,4]	[2,5]	[2,6]	[2,7]	[2,8]	[2,9]	[2,10]	[2,11]
(Zucker)	MG1	MG2	EH1	EH2	EH3	(leer)	L1	L2	L3	L4	L5
	0	5	k	g			R	e	w	e	
Zeile 3	[3,1]	[3,2]	[3,3]	[3,4]	[3,5]	[3,6]	[3,7]	[3,8]	[3,9]	[3,10]	[3,11]
(Eier)	MG1	MG2	EH1	EH2	EH3	(leer)	L1	L2	L3	L4	L5
	1	0	S	t	k		A	l	d	i	

Einkaufszettel (Beispiel):

Menge aus zwei Ziffernspalten: [MG1|MG2] = [1|2] -> 12

Einheit aus Buchstabenspalten: [EH1|EH2] = [k|g] -> "kg", [EH1|EH2|EH3] = [S|t|k] -> "Stk"

Sp.6 bleibt leer (visuelle Trennung zwischen Einheit und Laden)

Laden aus Buchstabenspalten: [L1|...|L5] = [E|d|e|k|a] -> "Edeka"

Hinweis: Leere Zellen (wie EH3 bei Mehl/Zucker, L5 bei Rewe/Aldi) bleiben einfach frei.

Beispiel: Bibliotheksliste (Nummern und Preis-Ziffern in einzelnen Zellen)

	Spalte 1		Spalte 2		Spalte 3		Spalte 4		Spalte 5		Spalte 6		Spalte 7		Spalte 8		Spalte 9	
	+		+		+		+		+		+		+		+		+	
Zeile 1	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]	[1,7]	[1,8]	[1,9]	[1,10]	[1,11]	[1,12]	[1,13]	[1,14]				
(Buch)	T1	T2	T3	T4	T5	T6	T7	Nr1	Nr2	Nr3	Nr4	Nr5	P1	P2	P3			
	M	a	t	h	e		7	9	7	8	3	1	1	9	9			
Zeile 2	[2,1]	[2,2]	[2,3]	[2,4]	[2,5]	[2,6]	[2,7]	[2,8]	[2,9]	[2,10]	[2,11]	[2,12]	[2,13]	[2,14]				
(Buch)	T1	T2	T3	T4	T5	T6	T7	Nr1	Nr2	Nr3	Nr4	Nr5	P1	P2	P3			
	P	h	y	s	i	k		9	3	4	1	0	2	2	4	9		

Bibliotheksliste (Beispiel):

Titel aus Buchstabenspalten: [T1|...|T7] = [M|a|t|h|e| |7] -> "Mathe 7" (Leerzeichen ist auch ein Zeichen!)

Nummer aus Ziffernspalten: [Nr1|...|Nr5] = [9|7|8|3|1] -> 97831

Preis aus Ziffernspalten: [P1|P2|P3] = [1|9|9] -> 1,99

Beispiel: Kalender (Datum als einzelne Ziffern pro Zelle)

	Spalte 1	Spalte 2	Spalte 3	Spalte 4	Spalte 5	Spalte 6	Spalte 7	Spalte 8
Zeile 1	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]	[1,7]	[1,8]
(Termin)	T_D1	T_D2	M_D1	M_D2	J_D1	J_D2	J_D3	J_D4
	2	3	0	5	2	0	2	6
Zeile 2	[2,1]	[2,2]	[2,3]	[2,4]	[2,5]	[2,6]	[2,7]	[2,8]
(Termin)	T_D1	T_D2	M_D1	M_D2	J_D1	J_D2	J_D3	J_D4
	1	1	0	6	2	0	2	6

Kalender (Beispiel):

Tag aus [T_D1|T_D2] = [2|3] -> 23

Monat aus [M_D1|M_D2] = [0|5] -> 05

Jahr aus [J_D1|J_D2|J_D3|J_D4] = [2|0|2|6] -> 2026

Gesamt: 23.05.2026

Diese tabellarische Organisation ist nur eine von vielen Möglichkeiten. Entscheidend dabei: **jedes Zeichen** – ob Buchstabe, Ziffer oder Leerzeichen – belegt genau eine Zelle. Das gilt nicht nur für Ziffern, sondern für alle Inhalte.

Wichtig ist: Jede Zelle hat eine eindeutige Adresse [Zeile, Spalte], sodass Programme die Informationen gezielt lesen und schreiben können.

Der Notizblock funktioniert damit wie der Speicher einer Maschine – **flexibel, übersichtlich und maschinenverständlich**.

3 Datentypen und Hilfsfunktionen – Was steht in einer Zelle?



Bisher haben wir Inhalte einfach in Zellen geschrieben, ohne genauer zu sagen, **was für eine Art von Inhalt** dort steht. In der Informatik ist das aber wichtig: Jede Zelle enthält einen Wert eines bestimmten **Datentyps**. Der Datentyp legt fest, welche Werte erlaubt sind und wie mit ihnen gearbeitet werden kann.

3.1 Einfache Datentypen (ein Zeichen pro Zelle)

Die drei grundlegenden Typen für eine einzelne Zelle sind:

- **char** (Zeichen): Eine Zelle enthält genau **einen** Buchstaben, eine Ziffer oder ein Symbol.
Beispiele: [1,1] = 'T', [1,2] = 'o', [3,3] = '?', [2,5] = ' ' (Leerzeichen ist auch ein char!)
- **digit** (Ziffer): Eine Zelle enthält genau **eine** Ziffer von 0 bis 9.
Beispiele: [1,1] = 1, [1,2] = 2, [2,1] = 0
Hinweis: digit ist ein Sonderfall von char – eine Ziffer ist auch ein Zeichen.
- **boolean** (Wahrheitswert): Eine Zelle enthält genau **einen** von zwei möglichen Werten: **ja** oder **nein** (bzw. **wahr** / **falsch**).
Beispiele: h[11,1] = **nein** (Merker „gefunden“), h[12,1] = **ja** (Merker „fertig“)
Typische Verwendung: Zustände und Merker im Hilfsblock.

3.2 Zusammengesetzte Typen (mehrere Zellen)

Einzelne chars und digits reichen oft nicht aus – wir wollen Wörter, Zahlen oder Kommazahlen speichern. Dafür gibt es **Hilfsfunktionen**, die mehrere Zellen lesen oder befüllen und den Inhalt als Ganzes übergeben. Sie sind sozusagen **Abkürzungen für wiederkehrende Abläufe** auf dem Notizblock.

- **string(zeile, spalte, zeichenkette)** – Zeichenkette schreiben:
Verteilt die angegebene Zeichenkette ab der Startzelle Zeichen für Zeichen auf aufeinanderfolgende Zellen.
string(zeile, spalteVon, spalteBis) – Zeichenkette lesen:
Liest alle Zellen von *spalteVon* bis *spalteBis* und gibt sie als Zeichenkette zurück.
Beim Lesen ist die Zeichenkette unbekannt – deshalb gibt man den Bereich explizit an.

```
string schreiben - string(1, 7, "Edeka"):
-> [1,7]='E', [1,8]='d', [1,9]='e', [1,10]='k', [1,11]='a'
```

```
string lesen - string(1, 7, 11):
-> liest [1,7] bis [1,11] -> "Edeka"
```

- **integer(zeile, spalte, zahl)** – Ganze Zahl schreiben:
Verteilt die angegebene Zahl ab der Startzelle Ziffer für Ziffer auf aufeinanderfolgende Zellen.
integer(zeile, spalteVon, spalteBis) – Ganze Zahl lesen:
Liest alle Zellen von *spalteVon* bis *spalteBis* und setzt sie zur vollständigen Zahl zusammen.

```
integer schreiben - integer(1, 8, 97831):
```

```
-> [1,8]=9, [1,9]=7, [1,10]=8, [1,11]=3, [1,12]=1
```

```
integer lesen - integer(1, 8, 12):  
-> liest [1,8] bis [1,12] -> 97831
```

- **float(zeile, spalte, zahl)** – Fließkommazahl schreiben:

Wie integer, aber im Notizblock steht auch das Komma als eigenes Zeichen.

In manchen Hilfsdarstellungen kann das Komma weggelassen werden; im eigentlichen Notizblock-Modell wird es aber mitgeschrieben.

float(zeile, spalteVon, spalteBis, kommaPosition) – Fließkommazahl lesen:

Liest alle Zellen von *spalteVon* bis *spalteBis* und setzt das Komma an der angegebenen Stelle wieder ein.

```
float schreiben - float(1, 13, 1,99):  
-> [1,13]=1, [1,14]=',', [1,15]=9, [1,16]=9
```

```
float lesen - float(1, 13, 16, 1):  
-> liest [1,13] bis [1,16], Komma nach Stelle 1 -> 1,99
```

```
float-Darstellung ohne Komma (vereinfachte Skizze):  
-> [1,13]=1, [1,14]=9, [1,15]=9
```

Hinweis:

Wir wollen mit den Zahlen auch rechnen können. Dies müssen wir dem Computer auch beibringen.

Vorerst stellen wir uns aber vor, dass wir mithilfe unseres eigenen Verstands mit den Zahlen auch rechnen können.

Erkenntnis:

Der Notizblock kennt selbst keine zusammengesetzten Werte – er enthält nur einzelne chars, digits oder booleans pro Zelle.

Wörter, Zahlen und Kommazahlen entstehen erst durch das **gezielte Lesen und Zusammensetzen mehrerer Zellen** mithilfe dieser Hilfsfunktionen.

Genau das werden wir später auch in JavaScript so umsetzen: Daten zeichenweise im Array ablegen und per Funktion auslesen.

4 Tätigkeitenblatt – Abläufe als Bausteine



Neben **Notizblock** (Daten) und **Hilfsblock** (Hilfsvariablen) ist ein **Tätigkeitenblatt** sinnvoll.

Dort stehen keine konkreten Werte, sondern **rezeptartige Abläufe**, die der Prozessor bei Bedarf ausführen kann.

Wozu dient das Tätigkeitenblatt?

- **Wiederverwendung:** Ein Ablauf wird einmal beschrieben und mehrfach genutzt.
- **Übersicht:** Lange Lösungen werden in kleine, verständliche Bausteine zerlegt.
- **Fehlersuche:** Man prüft gezielt einen Baustein statt das ganze Gesamtprogramm.

Möglicher Aufbau eines Eintrags:

- **Name der Tätigkeit** (z. B. *nummer_lesen*)
- **Startwerte/Parameter** (z. B. Zeile = 1)
- **Schritte** (rezeptartig, eindeutig)
- **Ergebnis** (z. B. zusammengesetzte Nummer im Notizblock)

Beispiel auf Basis der Bibliotheksliste:

Tätigkeit: `nummer_lesen(zeile)`

- 1) Setze `[10,1] = ""`.
- 2) Lies in der angegebenen Zeile die Spalten 2 bis 6.
- 3) Hänge jede gelesene Ziffer an `[10,1]` an.
- 4) Ergebnis: `[10,1]` enthält die vollständige Nummer.

Tätigkeit: `preis_lesen(zeile)`

- 1) Setze `[10,2] = ""`.
- 2) Lies in der angegebenen Zeile die Spalten 7 bis 9.
- 3) Hänge jede gelesene Ziffer an `[10,2]` an.
- 4) Ergebnis: `[10,2]` enthält den Preis ohne Komma (z. B. "199").

Didaktisch ist das der nächste logische Schritt: Lernende sehen, dass ein Programm nicht nur aus Daten besteht, sondern auch aus **wiederverwendbaren Tätigkeiten**. In einer späteren JavaScript-Umsetzung entsprechen diese Tätigkeiten dann typischerweise **Funktionen**.

5 Prozessor – der Ausführer



Der **Prozessor** (bspw. unser Gehirn) liest den Notizblock, führt Befehle aus und schreibt Ergebnisse zurück. Er kennt vier grundlegende Fähigkeiten: **Schreiben**, **Lesen**, **Falls-Abfrage** und **Schleife**.

5.1 1. Einfache Schreibvorgänge

Am einfachsten beginnt man mit reinen Schreibvorgängen.

Der Prozessor trägt also Werte an bestimmten Stellen in den Notizblock ein.

Beispiel auf Basis des Einkaufszettels:

- Trage für Mehl in Zeile 1, Spalte 1 die erste Ziffer der Menge ein: 1.
- Trage für Mehl in Zeile 1, Spalte 2 die zweite Ziffer der Menge ein: 2.
- Trage für Zucker in Zeile 2, Spalte 1 die erste Ziffer der Menge ein: 0.
- Trage für Zucker in Zeile 2, Spalte 2 die zweite Ziffer der Menge ein: 5.

Hier geht es nur darum zu verstehen: **Der Prozessor kann gezielt in einzelne Zellen schreiben.**

5.2 2. Einfache Lesevorgänge

Danach folgt das Lesen. Der Prozessor schaut an einer bestimmten Stelle nach, welcher Wert dort steht.

Beispiel auf Basis des Einkaufszettels:

- Lies bei Zucker die erste Mengenziffer aus Zeile 2, Spalte 1.
- Lies bei Zucker die zweite Mengenziffer aus Zeile 2, Spalte 2.
- Lies bei Eiern die Einheit aus Zeile 3, Spalte 3.
- Lies bei Mehl den Laden aus Zeile 1, Spalte 4.

Auch hier ist die Idee noch einfach: **Der Prozessor kann gezielt aus einzelnen Zellen lesen.**

5.3 3. Ein einfacher Durchlauf

Nun kann der Prozessor mehrere Einträge der Reihe nach abarbeiten.

Das ist bereits eine **Schleife**: Derselbe Ablauf wird für mehrere Zeilen und Zellen wiederholt.

Beispiel auf Basis der Bibliotheksliste:

Wir gehen jede einzelne Zeile und pro Zeile jede einzelne Zelle durch:

Beispiel eines einfachen Durchlaufs in natürlicher Sprache:

Für jede Zeile (= jedes Buch):

 Für jede Zelle der Zeile (= Titel, dann jede Ziffer der Nummer, dann jede Ziffer des Preises):

 Lies den Wert der aktuellen Zelle.

 Wenn wir uns in Sp. 2 bis 6 befinden, sammeln wir die Ziffern der Nummer.

 Wenn wir uns in Sp. 7 bis 9 befinden, sammeln wir die Ziffern des Preises.

So wird klar: **Der Prozessor kann mehrere Zeilen nacheinander bearbeiten – und innerhalb jeder Zeile Zelle für Zelle vorgehen.**

5.4 4. Eine Schleife mit Abfrage

Nun erweitern wir die Schleife um eine **Abfrage**.

Dabei wird beim wiederholten Durchlaufen zusätzlich geprüft, ob ein bestimmter Fall eingetreten ist.

Beispiel auf Basis der Bibliotheksliste:

Wir gehen wieder jede einzelne Zeile und pro Zeile jede einzelne Zelle durch:

Beispiel einer Schleife mit Abfrage in natürlicher Sprache:

Für jede Zeile (= jedes Buch):

 Für jede Zelle der Zeile (= Titel, dann jede Ziffer der Nummer, dann jede Ziffer des Preises):

 Lies den Wert der aktuellen Zelle.

 Wenn wir uns in Sp. 2 bis 6 befinden, sammeln wir die Ziffern der Nummer.

Wenn wir uns in Sp. 7 bis 9 befinden, sammeln wir die Ziffern des Preises.
Prüfe nach dem Einlesen der Zeile, ob die gelesene Nummer der gesuchten Nummer 97831 entspricht.
Falls ja, schreibe die Nummer und den Preis in die dafür vorgesehene Zeile 10.
(Auch dort müssen die Ziffern wieder spaltenweise in die einzelnen Zellen eingetragen werden.)
Falls nein, gehe zur nächsten Zeile weiter.

Hier kommen mehrere Fähigkeiten zusammen: **lesen**, **prüfen**, **gegebenenfalls schreiben** und **wiederholen** – und zwar auf zwei Ebenen: einmal über die Zeilen, einmal über die Zellen pro Zeile.

So wird zunächst in natürlicher Sprache deutlich, **was** der Prozessor tun soll.
Erst im nächsten Schritt wird daraus eine formale Programmiersprache.
Dadurch bleibt der Einstieg anschaulich und verständlich.

Die Didaktik soll darauf beruhen, dass die Schulmathematik mithilfe eines einfachen JavaScript-Modells mit Matrix als Notizblock beschrieben werden kann.

6 Rechnen mit integers – ein erster Algorithmus



In diesem Kapitel geht es nicht um einen Beweis, sondern um einen **Algorithmus**.

Wir wollen zeigen, wie man mit **integers** im Notizblock-Modell rechnen kann.

Das ist ein kleiner erster Schritt, um zu sehen: Schulmathematik kann durch die Funktionen einer Turing-Maschine (bzw. unserer Notizblock-Maschine) beschrieben werden – also durch **Lesen**, **Schreiben**, **Vergleichen** und **Springen**.

6.1 Grundidee

Wir legen die Ziffern **0 bis 9** im Speicher nebeneinander als Bereich an.

Dadurch kann der Prozessor innerhalb dieses Bereichs zählen, indem er von einer Zelle zur nächsten weitergeht.

Bei einer Aufgabe wie $2 + 5$ könnte man den Speicherbereich für die Ziffern als Zählbereich verwenden: Die **3. Spalte** (= die Ziffer 2) wird ausgelesen, und anschließend werden über eine Schleife **5 weitere Spalten** nacheinander gelesen – also ein wiederholtes Zählen von einer Zelle zur nächsten.

Für den Anfang betrachten wir nur **ganze Zahlen** ohne Komma.

6.2 Beispiel: Addition mit Übertrag

Wir addieren zwei integers, zum Beispiel 478 und 256.

Die Zahlen (478 und 256) stehen zeichenweise im Notizblock, und der Prozessor arbeitet von rechts nach links.

Genauso wie auch die einzelnen Ziffern (0 bis 9) im Notizblock vermerkt sind, sodass wir wie bereits erwähnt (Beispiel: $2 + 5$) wiederholt zählen können.

Jetzt müssen wir dies für jede Stelle der beiden Zahlen tun und dabei den Übertrag berücksichtigen.

A = 478

B = 256

Schritt 1: $8 + 6 = 14$

-> schreibe 4, merke Übertrag 1

Schritt 2: $7 + 5 + 1 = 13$
-> schreibe 3, merke Übertrag 1

Schritt 3: $4 + 2 + 1 = 7$
-> schreibe 7, kein weiterer Übertrag

Ergebnis: 734

Der Algorithmus benutzt also nur drei Dinge:

- **Lesen** der aktuellen Ziffern
- **Schreiben** der Ergebnisziffer
- **Merken** eines Übertrags im Hilfsblock

6.3 Beispiel: Subtraktion mit Übertrag / Ausleihen

Übung: Wie würde sich der Algorithmus anders beim Subtrahieren verhalten? Beachte dabei auch das wiederholte Zählen.

Hier sieht man bereits ein typisches Muster von Algorithmen:

- aktuelle Stelle ansehen
- Fall unterscheiden
- Hilfswert merken
- zur nächsten Stelle springen

6.4 Warum ist das ein Notizblock-Maschinen-Schritt?

Eine Notizblock-Maschine braucht keine magische Mathematik.

Sie arbeitet nur mit sehr einfachen Aktionen: lesen, schreiben, vergleichen und den nächsten Schritt auswählen.

Genau das machen wir hier auch.

Die Schulmathematik wird also nicht auf einmal anders – sie wird nur in kleine, ausführbare Schritte zerlegt.

6.5 Erkenntnis

Mit einem Notizblock und einem Prozessor kann man integers nicht nur speichern, sondern auch systematisch verarbeiten.

Das ist noch nicht die ganze Schulmathematik, aber ein wichtiger erster Baustein.

Später können wir darauf aufbauen, etwa mit Multiplikation, Division, Gleichungen und weiteren Verfahren.

7 Hilfsblock – Hilfsvariablen



Beim Programmieren braucht man oft **zusätzliche Variablen**, die nicht direkt zum Problem gehören, aber für die Berechnung notwendig sind. Beispiele: *Zähler* in Schleifen, *Zwischenergebnisse* bei Berechnungen oder *Flags*, um sich zu merken, ob etwas gefunden wurde.

Um diese **Hilfsvariablen** klar von den eigentlichen **Daten** zu trennen, nutzen wir ein zweites kariertes Papier – das **Hilfsblatt**. Zugriff erfolgt mit `h[Zeile, Spalte]` (h = Hilfsspeicher).

Typische Nutzfelder im Hilfsblatt:

- **Zähler und Zeiger** (z. B. Zeilen 1-5): „*Wo bin ich gerade?*“
Beispiel: Schleifenzähler, Index in einer Liste
- **Sammelstellen für Werte** (z. B. Zeilen 6-10): „*Was setze ich gerade zusammen?*“
Beispiel: Mehrere Ziffern zu einer Nummer oder einem Preis zusammensetzen
- **Zwischenwerte und Berechnungen** (z. B. Zeilen 6-10): „*Was berechne ich gerade?*“
Beispiel: Temporäre Summe, Zwischenergebnis bei Division
- **Zustände und Merker** (z. B. Zeilen 11-15): „*Was habe ich gefunden/erreicht?*“
Beispiel: Flag für "gefunden", aktuelles Minimum/Maximum
- **Werkzeuge und Konstanten** (z. B. Zeilen 16-25): „*Was brauche ich immer wieder?*“
Beispiel: Ziffern 0-9 für Grundrechenarten, mathematische Konstanten

Beispiel 1: Schleifenzähler beim Durchgehen einer Liste

	Sp.1	Sp.2	Sp.3	Sp.4	Sp.5	Sp.6	
Zeile 1	[1,1]	[1,2]	[1,3]	[1,4]	[1,5]	[1,6]	<- "Tobias" (6 Zeichen)
	T	o	b	i	a	s	
Zeile 2	[2,1]	[2,2]	[2,3]	[2,4]			<- "Erik" (4 Zeichen)
	E	r	i	k			
Zeile 3	[3,1]	[3,2]	[3,3]	[3,4]			<- "Leon" (4 Zeichen)
	L	e	o	n			

Jeder Buchstabe belegt eine eigene Zelle. Leere Zellen bleiben frei.

Wir wollen jede Zeile dieser Liste durchgehen.

Dazu müssen wir aber wissen, was die letzte Zeilennummer war, um mit der nächsten Zeile fortzufahren.

Daher speichern wir uns die aktuelle Zeilennummer im Hilfsspeicher und können nachher ab dieser fortsetzen.

Hilfsspeicher:

	Spalte 1
Zeile 1	1 -> d.h. wir sind bei Tobias

```

      | Spalte 1
-----+-----
Zeile 1 | 2 -> d.h. wir sind bei Erik

```

```

      | Spalte 1
-----+-----
Zeile 1 | 3 -> d.h. wir sind bei Leon

```

Beispiel 2: Eine Nummer aus einzelnen Ziffern zusammensetzen

In der Bibliotheksliste steht die Nummer nicht in einer einzigen Zelle, sondern verteilt auf mehrere Zellen. Während wir die Ziffern lesen, brauchen wir eine Hilfsvariable, in der wir die Nummer Schritt für Schritt zusammensetzen.

Bibliotheksliste, Zeile 1:

```

      | Spalte 2 | Spalte 3 | Spalte 4 | Spalte 5 | Spalte 6
-----+-----+-----+-----+-----+-----
Werte | 9           | 7           | 8           | 3           | 1

```

Hilfsspeicher:

```

      | Spalte 1
-----+-----
Zeile 6| nummer = ""

```

```

Lies Spalte 2 -> 9  -> nummer = "9"
Lies Spalte 3 -> 7  -> nummer = "97"
Lies Spalte 4 -> 8  -> nummer = "978"
Lies Spalte 5 -> 3  -> nummer = "9783"
Lies Spalte 6 -> 1  -> nummer = "97831"

```

Der Hilfsspeicher hält hier also fest, **welche Nummer bisher schon zusammengesetzt wurde**. Ohne diese Zwischenablage könnten wir die einzelnen gelesenen Ziffern nicht zu einer vollständigen Nummer verbinden.

Beispiel 3: Merken, ob etwas gefunden wurde

Wenn wir in der Bibliotheksliste die Nummer 97831 suchen, ist ein Merker hilfreich:

Hilfsspeicher:

```

      | Spalte 1
-----+-----
Zeile 11| gefunden = nein

```

Solange keine passende Nummer gefunden wurde:

```
gefunden = nein
```

Sobald die zusammengesetzte Nummer 97831 ist:

```
gefunden = ja
```

So kann sich der Prozessor merken, **ob die Suche erfolgreich war**, auch wenn er danach noch weitere Schritte ausführen muss.

Diese Trennung macht Programme **übersichtlicher** und hilft Anfängern zu verstehen, welche Daten zum Problem gehören und welche nur Hilfsmittel (Meta-Daten als Mittel zum Zweck) sind.

Frage: Fallen Dir weitere Beispiele ein, bei denen Hilfsvariablen nützlich sein könnten?

8 Laufzeitkomplexität: Schritte zählen



8.1 Warum Schritte zählen?

In der Informatik geht es oft darum, ein Problem zu lösen – zum Beispiel: **Finde eine bestimmte Person in einem Notizblock**.

Doch nicht jeder Lösungsweg ist gleich gut. Manche brauchen viele Schritte, andere kommen schneller ans Ziel.

Schritte zählen hilft uns dabei, verschiedene Lösungswege zu vergleichen:

- Wie viele Operationen braucht mein Programm mindestens? (Best Case)
- Wie viele im schlimmsten Fall? (Worst Case)
- Wie viele im Durchschnitt? (Average Case)

8.2 Beispiel 1: Lineare Suche im Notizblock

8.2.1 Startsituation:

- Notizblock mit n Einträgen: [Anna, Ben, Clara, David, Emma, Felix, Greta] (hier: $n = 7$)
- **Zu suchende Person:** Felix

8.2.2 Tätigkeit (rezeptartig):

1. Beginne beim ersten Eintrag.
2. Vergleiche den aktuellen Eintrag mit der **zu suchenden Person**.
3. Falls gefunden → fertig.
4. Falls nicht gefunden → gehe zum nächsten Eintrag.
5. Wiederhole, bis gefunden oder Notizblock zu Ende.

8.2.3 Schritte zählen:

- **Best Case:** Die **zu suchende Person** steht am Anfang → **1 Vergleich**
- **Worst Case:** Die Person steht am Ende oder fehlt → **n Vergleiche** (hier: 7)
- **Average Case:** Im Durchschnitt müssen wir die Hälfte durchsuchen → **n/2 Vergleiche**

Asymptotische Laufzeitordnung: $O(n)$ – „linear“

Je größer n wird, desto mehr Schritte brauchen wir proportional dazu.

8.3 Beispiel 2: Binärsuche (im alphabetisch sortierten Notizblock)

8.3.1 Startsituation:

- **Alphabetisch sortierter Notizblock:** [Anna, Ben, Clara, David, Emma, Felix, Greta] ($n = 7$)
- **Zu suchende Person:** Felix

8.3.2 Tätigkeit:

Wir brauchen zwei Marker im Hilfsspeicher: **linke Grenze** und **rechte Grenze**, die den aktuell noch zu durchsuchenden Bereich einschließen.

1. Setze linke Grenze = 1, rechte Grenze = 7 (= Anzahl der Einträge). (Beide Marker kommen in den Hilfsspeicher.)
2. Berechne die Mitte: (linke Grenze + rechte Grenze) / 2, abgerundet → das ist die aktuelle Prüfposition.
3. Lies den Eintrag an der aktuellen Prüfposition.
4. Ist dieser Eintrag die **zu suchende Person**? → fertig.
5. Kommt die **zu suchende Person** alphabetisch früher? → Setze rechte Grenze = aktuelle Prüfposition - 1.
6. Kommt sie alphabetisch später? → Setze linke Grenze = aktuelle Prüfposition + 1.
7. Wiederhole ab Schritt 2, solange linke Grenze $\leq$ rechte Grenze. (Falls nicht mehr: Person nicht vorhanden.)

Konkreter Ablauf für Felix:

- Durchlauf 1: linke Grenze = 1, rechte Grenze = 7 → Mitte = 4 → **David**. Felix > David → linke Grenze = 5.
- Durchlauf 2: linke Grenze = 5, rechte Grenze = 7 → Mitte = 6 → **Felix** → gefunden nach **2 Vergleichen**.

8.3.3 Schritte zählen:

- Bei jedem Schritt halbiert sich die Suchgröße.
- $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$
- Anzahl der Schritte: $\log_2(n)$

Beispiel: $n = 1024$ Personen

- Lineare Suche: bis zu 1024 Vergleiche
- Binärsuche: **10 Vergleiche** (weil $2^{10} = 1024$)

Asymptotische Laufzeitordnung: $O(\log n)$ – „logarithmisch“

Extrem effizient! Selbst bei sehr großen Notizblöcken braucht man nur wenige Schritte.

8.4 Vergleich der beiden Verfahren

Verfahren Worst Case Beispiel ($n=1000$ Personen)

Lineare Suche $O(n)$ 1000 Schritte

Binärsuche $O(\log n)$ 10 Schritte

Erkenntnis:

Je größer n wird, desto dramatischer ist der Unterschied zwischen den Verfahren.

Die asymptotische Laufzeitordnung (O -Notation) hilft uns, die Effizienz von Algorithmen zu verstehen – unabhängig von Hardware oder Programmiersprache.

8.5 Verbindung zur späteren Umsetzung

Auch in einer späteren JavaScript-Umsetzung mit Matrix als Notizblock können wir Schritte zählen, wenn wir auf unserem karierten Papier suchen:

- Jeder Funktionsaufruf ist ein Schritt
- Jeder Schreibzugriff ist ein Schritt
- Jede Abfrage mit Vergleich ist ein Schritt

Ein Programm zur linearen Suche hätte im Worst Case n Vergleiche.

Ein Programm zur Binärsuche hätte nur $\log_2(n)$ Vergleiche.

Fazit: Problemlösung ist Routine – aber manche Routinen sind effizienter als andere. Schritte zählen und asymptotische Laufzeitordnung helfen uns, die besten Werkzeuge auszuwählen.

TODOs: Weitere Beispiele für Laufzeitkomplexität, Übungen zum Schritte zählen, Einführung in O -Notation, Vergleich von Algorithmen, praktische Anwendungen, Verbindung zu späteren JavaScript-Implementierungen.

9 Sanfter Umstieg in eine Hochsprache (JavaScript)



Bis hierhin haben wir mit Notizblock-Adressen wie [Zeile, Spalte] gearbeitet.

Das ist didaktisch sehr hilfreich, weil man Speicher direkt sehen kann.

In JavaScript können wir ein Array-of-Arrays – eine **Matrix** – verwenden, um unser Notizblock-Modell direkt umzusetzen. So verlieren wir nicht die Sichtbarkeit und Nachvollziehbarkeit, sondern gewinnen die Kraft einer echten Programmiersprache hinzu.

9.1 Erster Übergang: Notizblock wird zur Matrix

Grundidee:

- Notizblock: [1,1] = 4

- JavaScript-Matrix: `notizblock[0][0] = 4;`

Beides meint dasselbe Prinzip: **ein Speicherplatz erhält einen Wert.**

Der Unterschied ist minimal – wir nutzen einfach die native Array-Struktur von JavaScript, um unser Notizblock-Konzept direkt abzubilden:

- Im Notizblock ist die Position sichtbar als `[Zeile, Spalte]`.
- In JavaScript bleibt das Konzept erhalten: `notizblock[zeile][spalte]`. Technisch beginnt die Indizierung bei 0.

9.2 Mini-Beispiel (gleicher Inhalt, zwei Schreibweisen)

Notizblock-Idee:

```
[1,1] = 4
[1,2] = 7
[1,3] = [1,1] + [1,2]
```

JavaScript-Idee (mit Matrix):

```
// 10x10 Matrix mit unabhängigen Zeilen (für den Anfang ist der konkrete Aufbau dieser Zeile erstmal nicht relevant)
let notizblock = Array.from({ length: 10 }, () => Array(10).fill(null));

notizblock[0][0] = 4;
notizblock[0][1] = 7;
notizblock[0][2] = notizblock[0][0] + notizblock[0][1];
```

Wichtig zu erkennen ist, dass wir im Gegensatz zu unserem Notizblock nicht mit 1 zu zählen beginnen, sondern mit 0.

9.3 Unsere Bücherei aus der alten Welt übertragen

```
// Altes Modell: zeichenweise gespeichert (wie auf dem karierten Notizblock)
let alt = [
  ['M','a','t','h','e',' ','7','9','7','8','3','1','1','9','9'],
  ['P','h','y','s','i','k',' ','9','3','4','1','0','2','2','4']
];

// Neues JS-Notizblock-Modell: Matrix + bereits zusammengefasste Werte
let notizblock = Array.from({ length: 10 }, () => Array(10).fill(null));

// Zeile 0 = erstes Buch, Zeile 1 = zweites Buch
// Spalte 0 = Titel, Spalte 1 = Nummer, Spalte 2 = Preis
notizblock[0][0] = 'Mathe 7';
notizblock[0][1] = 97831;
notizblock[0][2] = 1.99;

notizblock[1][0] = 'Physik';
```



```
notizblock[1][1] = 93410;
notizblock[1][2] = 22.49;
```

9.4 Schleifendenken vom Notizblock zu JavaScript

```
// Wir haben die Werte bereits zusammengefasst.
// Deshalb sammeln wir nicht mehr Zeichen für Zeichen.
// Eine innere Schleife ist hier dadurch nicht mehr nötig.

for (let zeile = 0; zeile <= 1; zeile++) {      // jedes Buch
    let titel = notizblock[zeile][0];
    let nummer = notizblock[zeile][1];
    let preis = notizblock[zeile][2];

    console.log('Zeile', zeile, 'Titel:', titel, 'Nummer:', nummer, 'Preis:', preis); // Ausgabe der Werte
}
```

Analog zum Notizblock bleibt die Denkweise gleich: Wir gehen alle Bücher der Reihe nach durch. Da die Zellen bereits zusammengefasste Werte enthalten (String/Number), entfällt hier die innere Schleife über Einzelzeichen. Didaktischer Nutzen: Lernende erkennen, dass JavaScript nichts Neues schafft. Die Programmiersprache gibt nur die technische Umsetzung unseres Notizblock-Konzepts. Das Modell – adressierter Speicher – bleibt erhalten.

Hinweis: Bei unserem Notizblock-Modell haben wir noch zwischen **Notiz-** und **Hilfsblock** unterschieden (Daten vs. Hilfsvariablen). In JavaScript nutzen wir für beide Kontexte einfach Variablen. Der konzeptionelle Unterschied bleibt in unserem Denken – aber die technische Umsetzung ist jetzt vereinheitlicht.

10 Zusammenhang Informatik - Mathematik



In Bearbeitung ...

Die Schulmathematik – Arithmetik, Algebra, Analysis, Stochastik, Statistik – scheint nur bedingt was mit Informatik zu tun zu haben. Dennoch lassen sich alle diese Probleme prinzipiell auf einer Registermaschine lösen.

Die Idee dahinter:

- Eine Registermaschine hat nur einen Speicher (die „Register“) und elementare Befehle wie Lesen, Schreiben, Vergleichen und Springen.
- Komplexe Rechenoperationen wie Addition, Subtraktion, Multiplikation oder Division lassen sich Schritt für Schritt aus diesen einfachen Befehlen aufbauen.
- Variablen der Mathematik entsprechen dabei Speicheradressen (Register), Bedingungen und Schleifen werden über Vergleichs- und Sprungbefehle umgesetzt.

Mächtigkeit: Eine Registermaschine ist Turing-vollständig – das bedeutet, dass alles, was algorithmisch lösbar ist, auf ihr dargestellt werden kann. Damit kann die gesamte Schulmathematik ausgeführt werden.

10.0.1 Gleichungen über Zahlen

Ein typisches Beispiel aus der Schulmathematik ist das Lösen von Gleichungen:

- Funktional (algebraisch): $x + 3 = 7 \rightarrow x = 7 - 3$
- Imperativ / programmatisch: Schrittweise Werte testen oder hochzählen, bis die Gleichung erfüllt ist.

Beide Paradigmen können auf Registermaschinen abgebildet werden – entweder durch Umkehrfunktionen oder durch schrittweise Verarbeitung mit Funktionsaufrufen und Vergleichen.

10.0.2 Gleichungen über nicht-numerische Objekte: Farben

Gleichungen müssen nicht nur mit Zahlen gelöst werden. Wir können auch andere Objekte betrachten, z. B. Farben.

- Funktion **f**: vertauscht Rot und Cyan, Gelb und Blau, Grün und Magenta etc.
- Umkehrfunktion **f-1**: kehrt die Vertauschung wieder um.

Beispiel:

```
f(f(x)) = gelb | f-1 auf beide Seiten
f(x)      = f-1(gelb) = blau | f-1 erneut
x          = f-1(blau) = gelb
```

Ergebnis: $x = \text{gelb}$

Dieses Beispiel zeigt, dass Gleichungen auch über nicht-numerische Objekte gelöst werden können, solange die Funktion bijektiv ist.

10.0.3 Gleichungssysteme über bewegende Objekte

Betrachten wir zwei Autos auf einer Straße, die unterschiedliche Startpositionen und Geschwindigkeiten haben. Wir möchten bestimmen, wann sie sich treffen.

1. Mathematisch-algebraisch

- Auto A: $\text{WegA}(t) = v_A * t + s_{A,0}$
- Auto B: $\text{WegB}(t) = v_B * t + s_{B,0}$
- Gleichungssystem: $\text{WegA}(t) = \text{WegB}(t)$
- Lösung über Umkehrfunktionen: $t = (s_{B,0} - s_{A,0}) / (v_A - v_B)$

2. Imperativ / programmatisch

- Wir starten bei $t = 0$ und erhöhen die Zeit schrittweise (hochzählen).
- In jedem Schritt wird $\text{WegA}(t)$ und $\text{WegB}(t)$ berechnet.
- Wenn $\text{WegA}(t) = \text{WegB}(t)$, haben wir die Lösung gefunden.
- Dies entspricht dem klassischen Programmierparadigma, wie wir es z. B. in einer einfachen JavaScript-Umsetzung mit Schleifen formulieren könnten.

Dieses Beispiel illustriert:

- Gleichungssysteme können sowohl algebraisch über Umkehrfunktionen gelöst werden als auch imperativ durch Schritt-für-Schritt-Berechnung.
- Die imperativen Schritte lassen sich auf eine Registermaschine übertragen, da jeder Schritt nur elementare Befehle benötigt (Werte lesen, schreiben, vergleichen, springen).
- So wird deutlich, dass Registermaschinen oder eine einfache JavaScript-Umsetzung auch dynamische, zeitabhängige Probleme modellieren können.
- Damit können sowohl funktionale Lösungen über Umkehrfunktionen als auch imperative Lösungen über schrittweises Abarbeiten beschrieben werden.

Für Lernende bedeutet dies: Registermaschinen und eine einfache JavaScript-Umsetzung mit Matrix sind universelle Werkzeuge, die weit über einfache Zahlenoperationen hinausgehen. Sie können auch Aufgaben lösen, die abstrakte Strukturen oder zeitabhängige Systeme betreffen, solange diese als Speicherinhalte modelliert werden.

11 Schlussfolgerungen



11.1 Was ist eine Schlussfolgerung?

Eine **Schlussfolgerung** ist ein logischer Ableitungsschritt, der neue Aussagen direkt aus bestehenden Prämissen ableitet. In komplexen Beweisen kann eine Schlussfolgerung aus mehreren aufeinander aufbauenden Einzelschritten bestehen. Jede Tei ableitung ist dabei eine eigene Schlussfolgerung.

In vielen Beispielen werden Mengenlogik und Vergleiche angewandt, um Schlussfolgerungen zu ziehen. Eigentlich kann man aber mit jedem deterministischen System oder Modell Schlussfolgerungen betreiben.

11.2 Beispiele aus der natürlichen Sprache

Beispiel 1 – Tiere:

- Prämisse 1: Kein Tier ist müde.
- Prämisse 2: Einige Tiere sind nicht hungrig.
- Schlussfolgerung: Mindestens einige Tiere sind nicht müde.

Beispiel 2 – Sokrates:

- Prämisse 1: Alle Menschen sind sterblich.
- Prämisse 2: Sokrates ist ein Mensch.

- Schlussfolgerung: Sokrates ist sterblich.

Erkenntnis:

- Schlussfolgerungen sind die Bausteine logischen Denkens.
- Sie können auf Mengen, Eigenschaften oder konkrete Objekte angewendet werden.
- Auch ohne Zahlen lassen sich daraus neue Aussagen ableiten.

12 Beweisführung



12.1 Was ist eine Beweisführung?

Eine **Beweisführung** ist ein geordneter Prozess (ein Spezialfall der Problemlösung), um eine Behauptung oder These systematisch aus bekannten Fakten oder Prämissen zu zeigen. Sie besteht aus drei Kernbestandteilen:

- **Behauptung** / **These**: Das Ziel, das bewiesen werden soll.
- **Prämissen**: Bekannte Fakten, Annahmen oder Axiome, die als Ausgangspunkt dienen.
- **Werkzeuge**: Anwendung von Schlussfolgerungen oder andere logische/arithmetic Methoden, z. B. direkter Beweis, Beweis durch Widerspruch oder Induktion.

12.2 Beispiel 1 – Sokrates

Behauptung: Sokrates ist sterblich. (*Die Aufstellung der Behauptung startet die Beweisführung.*)

- Prämisse 1: Alle Menschen sind sterblich.
- Prämisse 2: Sokrates ist ein Mensch.
- Schlussfolgerung ist hier ein Werkzeug: Sokrates ist sterblich.

Erkenntnis: Der Großteil der Beweisführung besteht in der Anwendung von Schlussfolgerungen. Die Behauptung gibt den Rahmen vor.

12.3 Beispiel 2 – Arithmetik: Quadratische Faktorisierung

Behauptung: Für jede Zahl a gilt $a^2 - 1 = (a-1)(a+1)$.

12.3.1 einige arithmetische Werkzeuge

- Ausklammern / Distributivgesetz: $a(b+c) = ab + ac$
- Faktorisierung von Summen/Differenzen: $a^2 - b^2 = (a-b)(a+b)$
- Potenzgesetze: $a^m \cdot a^n = a^{m+n}$
- Umformung von Termen: z. B. $a^2 - 12 \rightarrow a^2 - 1$
- Substitution: Setze bekannte Werte ein (z. B. $b=1$)
- Induktionswerkzeuge: Für wiederkehrende Strukturen (z. B. Summenformeln)
- Übersetzen in arithmetische Sprache
- Polynom-Kongruenz
- aus einer unendlich zu testenden Menge eine endliche herbeiführen (Resteklassen, Transitivität)

12.3.2 Ausgewähltes Werkzeug

Für diese Beweisführung wählen wir das Werkzeug **Faktorisierung von Differenzen: $a^2 - b^2 = (a-b)(a+b)$** - ein Spezialfall des Werkzeugs **Schablonenerkennung**.

12.3.3 Schlussfolgerung / Ableitung

1. Faktorisierung von Differenzen: $a^2 - b^2 = (a-b)(a+b)$
2. Setze $b = 1$ ein: $a^2 - 1^2 = (a-1)(a+1)$
3. Da $1^2 = 1$, folgt: **$a^2 - 1 = (a-1)(a+1)$** .

12.4 Erkenntnis

- Beweisführung ist ein geordneter Rahmen, in dem Schlussfolgerungen als Werkzeuge eingesetzt werden.
- Schlussfolgerungen können einzeln oder in Ketten auftreten.
- Arithmetische Beweise nutzen Werkzeuge wie Faktorisierung, Potenzgesetze oder Summenregeln.
- Selbst scheinbar „magische“ Formeln werden nachvollziehbar, wenn Behauptung, ausgewähltes Werkzeug und Ableitung klar definiert sind.

Fazit: Wer Behauptung, Werkzeuge und Ableitung kennt, kann jede Beweisführung Schritt für Schritt aufbauen und verstehen.

12.5 Beispiel 3 - Teilbarkeit prüfen mit Polynom-Kongruenz

Behauptung (natürliche Sprache): **$2n^2 - 3n + 6$ ist durch 4 teilbar.**

Stimmt diese Behauptung für alle natürlichen Zahlen n ?

12.5.1 Werkzeuganwendung: Übersetzen in arithmetische Sprache

"Durch 4 teilbar" übersetzen wir in arithmetische Sprache:

$$2n^2 - 3n + 6 \equiv 0 \pmod{4}$$

12.5.2 Werkzeuganwendung: Polynom-Kongruenz / Restklassen (Strukturerhalt des Terms)

Vorweg: Was bedeutet Kongruenz für mod 4? $n \equiv r \pmod{4}$ genau dann, wenn $n = r + 4 \cdot m$ für ein ganzzahliges m .

Strukturgleichheit bei Modulo-Rechnungen

- $5 \equiv 9 \pmod{4} \rightarrow$ beide Zahlen Rest 1 $\rightarrow$ gleiche Restklasse.
- $t(n) = n, t'(n) = n$
 - $t(5) = 5, t'(9) = 9$
 - $5 \equiv 9 \pmod{4} \rightarrow$ beide Restklasse 1, Struktur bleibt parallel.
- $t(n) = 2n, t'(n) = 2n$
 - $t(5) = 10, t'(9) = 18$
 - $5 \equiv 9 \pmod{4}$
 - $10 \equiv 18 \pmod{4} \rightarrow$ beide Restklasse 2, Struktur bleibt parallel.

Wie bei diesen Modulo-Rechnungen erkennbar, kann man nur anhand von n (ohne die Terme zu kennen) die Restklasse für mod 4 bestimmen. Anders verhält es sich bei diesem Beispiel:

- $t(n) = 2n, t'(n) = 3n$
 - $t(5) = 10, t'(9) = 27$
 - obwohl $5 \equiv 9 \pmod{4}$, ist $10 \not\equiv 27 \pmod{4}$

Hier unterscheidet sich die Restklasse zwischen den bloßen n -Werten und der Ausführung der Terme.

Die Struktur bleibt hier also nicht erhalten.

Merksatz: Wenn die Struktur gleich bleibt, genügt es, die Variable durch ihren Vertreter der Restklasse zu ersetzen anstelle des gesamten Terms.

In unserem Fall geht es darum zu wissen, wann der Term durch 4 teilbar ist.

Folgendes funktioniert für $2n^2 - 3n + 6 \equiv 0 \pmod{4}$ leider nicht, weil dadurch der Term nicht mehr bekannt ist:

- $0 \bmod 4 = 0$
- $1 \bmod 4 = 1$
- $2 \bmod 4 = 2$
- ...

ABER: Wir wissen ja, dass wir n zum Testen verwenden können ohne dass die Struktur verloren geht.

12.5.3 Beweisführung: Prüfung aller Restklassen

Wir prüfen jede Restklasse $n \equiv r \pmod{4}$:

- $n \equiv 0 \pmod{4} \rightarrow 2 \cdot 0^2 - 3 \cdot 0 + 6 = 6 \rightarrow 6\%4 = 2 \rightarrow$ nicht teilbar
- $n \equiv 1 \pmod{4} \rightarrow 2 \cdot 1^2 - 3 \cdot 1 + 6 = 5 \rightarrow 5\%4 = 1 \rightarrow$ nicht teilbar
- $n \equiv 2 \pmod{4} \rightarrow 2 \cdot 2^2 - 3 \cdot 2 + 6 = 8 \rightarrow 8\%4 = 0 \rightarrow$ teilbar
- $n \equiv 3 \pmod{4} \rightarrow 2 \cdot 3^2 - 3 \cdot 3 + 6 = 15 \rightarrow 15\%4 = 3 \rightarrow$ nicht teilbar

Frage: Warum brauchen wir nur die Restklassen testen und nicht alle Zahlen?

Zur Erinnerung: Modulo gibt den Rest einer Division aus. (in unserem Beispiel zwischen einschließlich 0 und 3)

Auch wenn wir ein $n > 3$ testen, bekommen wir auch nur ein Modulo-Ergebnis (einen Rest) zwischen einschließlich 0 und 3.

Nur im Falle der Restklasse 0 wissen wir, dass es sich um eine Zahl handelt, die durch 4 teilbar ist.

Fazit: Nur für $n \equiv 2 \pmod{4}$ (also bspw. für $n = 2, 6, 10, \dots$) ist der Term durch 4 teilbar.

Mit den Werkzeugen **Übersetzen in arithmetische Sprache** und **Polynom-Kongruenz** konnten wir die Teilbarkeit vollständig nachvollziehen und ohne aufwendige algebraische Tricks prüfen.

12.6 Beispiel 4 - Prüfung der Geradzahligkeit

Ausgangsbehauptung: **Wenn $2n^2 - 3n + 6$ durch 4 teilbar ist (siehe Beispiel 3), dann ist n gerade.**

Übung von: Lehrerfortbildung Baden-Württemberg – Übungen zum Beweisen

Aus der Restklassen-Analyse in Beispiel 3 wissen wir: Nur $n \equiv 2 \pmod{4}$ liefert Teilbarkeit durch 4.

12.6.1 Geradzahligkeit prüfen mit Modulo 2

Eine Zahl ist gerade, genau dann, wenn $n \bmod 2 = 0$.

Wir setzen die Restklasse für die Teilbarkeit ein. Wichtig: Wir müssen **nicht alle Zahlen** prüfen, sondern nur die Vertreter der Restklassen. (siehe Polynom-Kongruenz), nämlich $n \equiv 2 \pmod{4}$.

$$(2 \cdot 2^2 - 3 \cdot 2 + 6) \bmod 2 = (8 - 6 + 6) \bmod 2 = 8 \bmod 2 = 0$$

Fazit: Das Ergebnis ist 0 $\rightarrow$ die Zahl ist gerade. Damit gilt die Behauptung.

Mit **Modulo 4** haben wir die Teilbarkeit ermittelt, mit **Modulo 2** die Geradzahligkeit überprüft. Dank der Restklassenanalyse ist das Verfahren übersichtlich und vollständig.

13 Tipps zur Didaktik



- Bei Übungen ist es wichtig, dass der Lernende seinen eigenen Weg nutzt, um zum Ziel zu kommen. Der Lehrende muss erkennen, dass trotzdem der Weg unkonventionell und ggf. auch umständlich ist, dieser auch zum Ziel führen könnte. Beispiel: Wenn der Schüler für den Anfang folgendes definiert: `var produktUndPreis = 'Brot für 1,99€'` und er das Produkt und den Preis ermitteln möchte, dann kann er das trotzdem herausfinden, auch wenn Produkt und Preis nicht in getrennte Variablen ausgewiesen ist.
- Die Erklärungen müssen häppchenweise (modular) stattfinden, sodass jeder das Thema nachvollziehen kann.
- Da Problemlösung auch nur eine Routine und keine Magie ist, kann jeder die zu lernenden Themen nicht nur logisch verstehen, sondern auch kreativ und problemlösend in diesem Bereich agieren.

- Wenn man ein Konzept erlernt, müssen erst die dazugehörigen Fundamente verstanden werden. (viele WARUM-Fragen müssen gestellt werden)
 - Das bedeutet, es muss vom Konzept aus ein roter Faden bis hin zum Fundament verfolgt werden.
 - Beim Lösen von Aufgaben aus dem Konzept sollte man im Laufe der Zeit trotzdem ein gedankliches Framework entwickeln.
 - Das bedeutet, dass man beim Bearbeiten der Aufgabe nicht vom Fundament aus bis zum Konzept über die Zusammenhänge nachdenkt, sondern das gedankliche Framework beinhaltet die Struktur und die wichtigsten Tipps des Fundaments.
 - Damit kann man effizienter und zielgerichteter arbeiten, da das Framework als Leitfaden dient.
- Gut wäre es, ein Thema (bspw. Stochastik) über ein Fundament (bspw. Kombinatorik in Baumstruktur) zu vermitteln. Auch u.a. das mehrfache Falten von Papier kann auf Kombinatorik reduziert werden.

14 Wie kann man das Kochen lernen?



- Dieses Thema passt hier gut rein, weil dies ein gutes Beispiel ist, um meine Philosophie des Lernens über einen roten Faden zu verinnerlichen.
- Wirklich kochen kann man, wenn man keine Rezepte braucht, weil man sich ein gutes Rezept selber durch die physikalischen, biochemischen Grundlagen und durch Kenntnisse über die geschmackliche Komposition von Gewürzen und anderen Lebensmittel ableiten kann.
- Hat man diese Grundlagen einmal verstanden und häufig angewendet, entwickelt sich daraus ein mentales Framework. Dieses Framework enthält die wichtigsten Zusammenhänge und Erfahrungswerte in verdichteter Form. Dadurch muss man beim Kochen nicht jedes Mal alle physikalischen und biochemischen Prozesse bewusst herleiten, sondern kann schnell und zielgerichtet entscheiden.